Executive summary



- Experiment with different compilation targets for Remora.
- Remora with PyTorch ran **5.4% faster** than native Darknet in our comparison.

Experiment 1: original pipeline



- Start with YOLOv4 written in Remora. **OG** denotes the original source.
- Compile through Futhark to CUDA and run on one H200 GPU.
- Compare against native Darknet C++.

Remora source: `examples/darknet/yolov4.remora`

Commit `72510c69006b90d1d5bcd1c472a69212b89b3116`, August 17, 2026.

Original pipeline: results

Pipeline	Time (ms)
Remora (OG) $\rightarrow$ Futhark	911.595
Darknet C++	9.364

- Original Remora through Futhark is about 100$\times$ slower.

Can we improve the Remora code?

Experiment 2: optimized Remora



- **MD** is our modified Remora source in `remora-copy`.
- Replace `reduce` with `reduce/zero` in dot products, giving the sum an explicit starting value of zero.
- This exposes the reduction in a form the compiler can optimize more effectively.

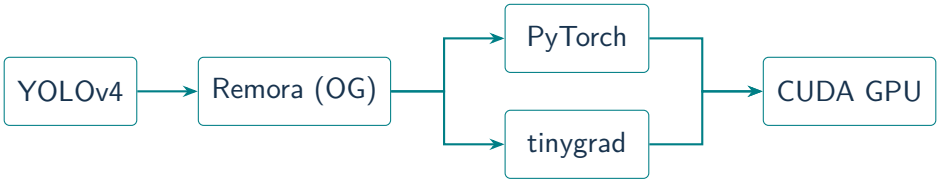
Optimized Remora: results

Pipeline	Time (ms)
Remora (OG) $\rightarrow$ Futhark	911.595
Remora (MD) $\rightarrow$ Futhark	482.714
Darknet C++	9.364

- About 1.9$\times$ faster than the original Remora pipeline.
- Still about 52$\times$ slower than Darknet.

What if we try a different backend?

Experiment 3: alternative backends



- Built a small, specialized compiler for Remora YOLOv4 as a **rapid prototype**.
- Read the Remora AST and map supported definitions to high-level PyTorch or tinygrad operations.
- Establish an **end-to-end** comparison using existing backend operations, without advanced compiler techniques or SIMD/MIMD analysis.

Alternative backends: results

Pipeline	Time (ms)
Remora (OG) $\rightarrow$ PyTorch	8.862
Direct PyTorch	8.927
Remora (OG) $\rightarrow$ tinygrad	156.294
Direct tinygrad	156.301
Darknet C++	9.364

- Remora with PyTorch is **5.4% faster** than Darknet. Both backends closely match their direct implementations.
- Rapid prototyping gives us an end-to-end baseline for future Futhark/MLIR compilation and scheduling work.
- We checked all pipelines for correctness. After NMS, all kept the same candidate boxes and classes on the tested inputs.

Next semester

- Explore **AI-enabled compilation**: use AI to investigate which compiler technologies suit a language and its workloads.
- Use AI to help build compilers, extending our Remora backend prototype.
- Use AI as a separate pass to generate or improve compiled code for specific programs, building on experiments such as `reduce/zero`.
- Explore a general framework for these approaches with MLIR and other compilation targets.